

Phase 3 – Pods, Deployments & Scaling (Amazon EKS)

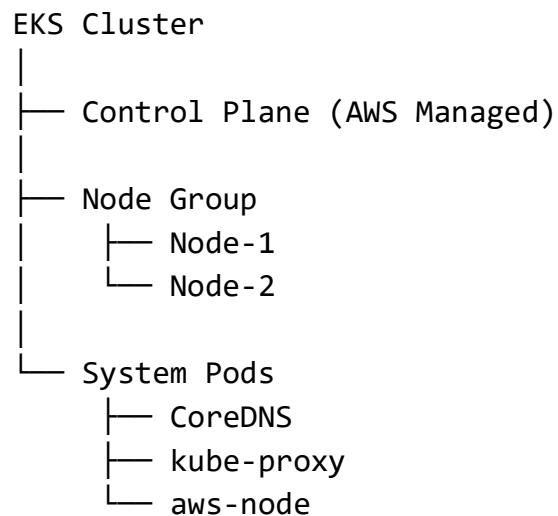
Objective

In this phase we learn:

- ✓ What a Pod is
- ✓ How Kubernetes schedules Pods
- ✓ How to inspect Pods
- ✓ How to access running containers
- ✓ How to view logs
- ✓ Why Pods are not used directly in production
- ✓ What Deployments and ReplicaSets are
- ✓ How self-healing works
- ✓ How scaling works
- ✓ How desired state works

Architecture Before Phase 3:

At the end of Phase 2 we had:



Verified using:

```
kubectl get nodes  
kubectl get pods -A
```

Example Output:

NAME	STATUS
ip-172-31-17-46.ap-south-1.compute.internal	Ready
ip-172-31-40-167.ap-south-1.compute.internal	Ready

Lab 1 – Create First Pod

Create YAML File

File:

nginx-pod.yaml

Content:

```
apiVersion: v1  
kind: Pod  
  
metadata:  
  name: nginx-pod  
  
spec:  
  containers:  
    - name: nginx  
      image: nginx:latest  
      ports:  
        - containerPort: 80
```

Understanding The YAML

Field

apiVersion
kind
metadata.name
spec

Purpose

Kubernetes API version
Resource type
Pod name
Desired configuration

containers
image
containerPort

Container list
Container image
Application port

Deploy Pod

Command:

```
kubectl apply -f nginx-pod.yaml
```

Output:

```
pod/nginx-pod created
```

Screenshot Reference

```
PS> kubectl apply -f nginx-pod.yaml
```

```
pod/nginx-pod created
```

Verify Pod Creation

Command:

```
kubectl get pods
```

Output:

NAME	READY	STATUS
nginx-pod	1/1	Running

Screenshot Reference

```
PS> kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-pod	1/1	Running	0	5s

Check Pod Scheduling

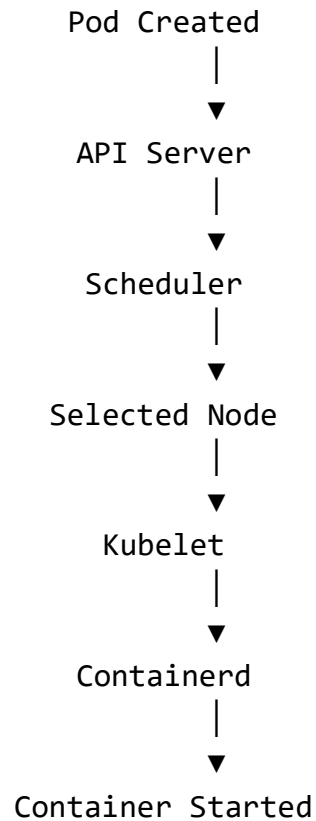
Command:

```
kubectl get pods -o wide
```

Output:

NAME	READY	STATUS	IP	NODE
nginx-pod	1/1	Running	172.31.19.121	ip-172-31-17-46

What Happened Internally?



Important Observation

Node IP:

172.31.17.46

Pod IP:

172.31.19.121

They are different.

Kubernetes gives each Pod its own IP.

PS> kubectl get pods -o wide

NAME	READY	STATUS	IP	NODE
nginx-pod	1/1	Running	172.31.19.121	ip-172-31-17-46

Inspect Pod

Command:

```
kubectl describe pod nginx-pod
```

Useful Sections:

Node

IP

Image

Container ID

Events

Conditions

Volumes

Most Important Section

Events:

Scheduled

Pulling

Pulled

Created

Started

Example:

Successfully assigned default/nginx-pod
to ip-172-31-17-46

This proves Scheduler selected the node.

Events:

Scheduled

Pulling

Pulled

Created

Started

View Pod Logs

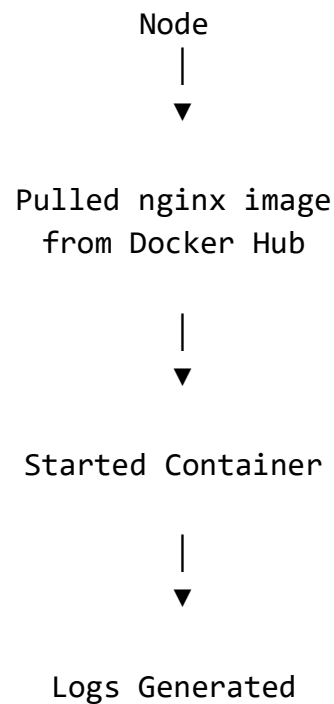
Command:

```
kubectl logs nginx-pod
```

Output:

```
nginx/1.31.1  
Configuration complete  
ready for start up
```

What Happened?



How Did Kubernetes Find Docker Hub?

Image specified:

```
image: nginx:latest
```

Kubernetes automatically resolves it as:

```
docker.io/library/nginx:latest
```

Default Registry:

Docker Hub

Enter Running Container

Command:

```
kubectl exec -it nginx-pod -- /bin/bash
```

Output:

```
root@nginx-pod:/#
```

Important Concept

Command targets:

Pod

but executes inside:

Container

Multi-Container Pods

Example:

```
multi-pod
```

```
|— nginx
|— busybox
```

Access specific container:

```
kubectl exec -it multi-pod -c nginx -- /bin/bash
```

or

```
kubectl exec -it multi-pod -c busybox -- /bin/sh
```

List Containers Inside Pod

Command:

```
kubectl get pod nginx-pod -o jsonpath="{.spec.containers[*].name}"
```

Output:

```
nginx
```

Delete Pod

Command:

```
kubectl delete pod nginx-pod
```

Output:

```
pod "nginx-pod" deleted
```

Verify:

```
kubectl get pods
```

Output:

```
No resources found
```

Important Learning

Standalone Pods:

Delete Pod

|



Gone Forever

No self-healing.

Lab 2 – Deployments

Why?

Because production workloads should never run as standalone Pods.

Create Deployment YAML

File:

```
nginx-deployment.yaml
```

Content:

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
  name: nginx-deployment

spec:
  replicas: 1

  selector:
    matchLabels:
      app: nginx

  template:
    metadata:
      labels:
        app: nginx

    spec:
      containers:
        - name: nginx
          image: nginx:latest
```

Understanding Deployment

Deployment



ReplicaSet



Pod



Container

Deploy

Command:

```
kubectl apply -f nginx-deployment.yaml
```

Verify:

```
kubectl get deployments  
kubectl get replicaset  
kubectl get pods
```

Deployment



ReplicaSet



Pod

Self-Healing Demonstration

Get Pod:

```
kubectl get pods
```

Delete Pod:

```
kubectl delete pod <pod-name>
```

Watch:

```
kubectl get pods -w
```

Immediately:

Old Pod Deleted

New Pod Created

What Happened Internally?

Before:

Desired Pods = 1

Actual Pods = 1

After Deletion:

Desired Pods = 1

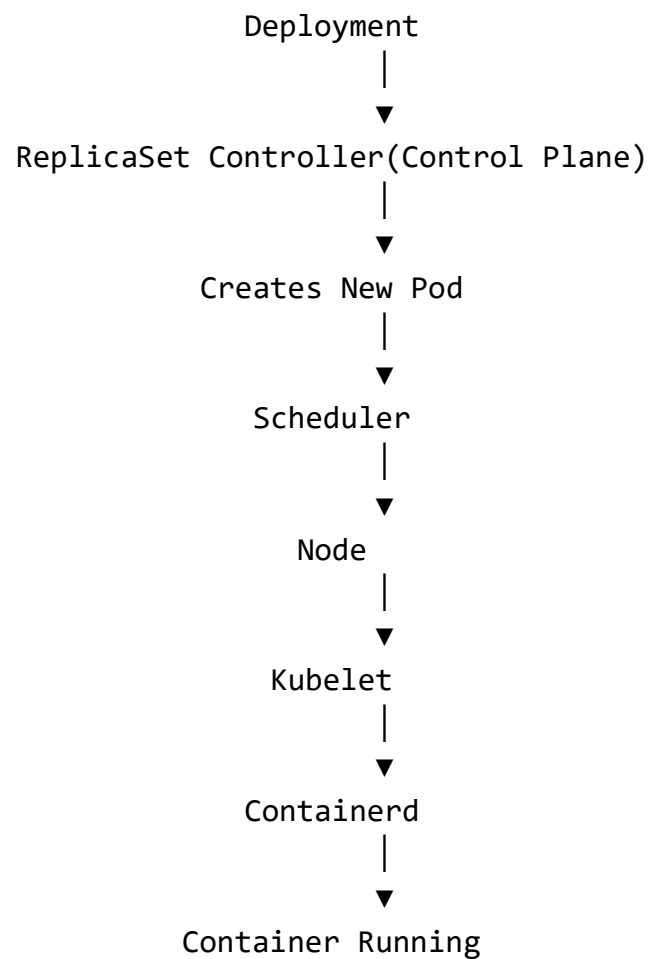
Actual Pods = 0

ReplicaSet notices mismatch:

Desired != Actual

Creates replacement Pod.

Backend Components Involved



Important Clarification

ReplicaSet is NOT running on worker nodes.

ReplicaSet logic runs inside:

kube-controller-manager

which is part of:

EKS Control Plane

managed by AWS.

Lab 3 – Scaling

Current Deployment:

Replicas = 1

Scale Up

Command:

```
kubectl scale deployment nginx-deployment --replicas=3
```

Verify:

```
kubectl get pods
```

Output:

3 Pods Running

Backend Process

Before:

Desired = 1

Actual = 1

After Scale:

Desired = 3

Actual = 1

ReplicaSet creates:

Pod-2

Pod-3

```
kubectl scale deployment nginx-deployment --replicas=3
```

3 Pods Running

Scale Down

Command:

```
kubectl scale deployment nginx-deployment --replicas=1
```

Result:

ReplicaSet terminates extra Pods

Output:

1 Pod Running

Kubernetes Desired State

This is the biggest lesson of Phase 3.

You are not telling Kubernetes:

Create Pod

You are telling Kubernetes:

I want 3 Pods.

Kubernetes continuously ensures:

Desired State

=

Actual State

Key Takeaways

- ✓ Pod is the smallest deployable unit in Kubernetes
- ✓ Pod contains one or more containers
- ✓ Scheduler selects a node
- ✓ Every Pod gets its own IP
- ✓ Logs are viewed using `kubectl logs`
- ✓ Containers can be accessed using `kubectl exec`
- ✓ Standalone Pods are not self-healing
- ✓ Deployments manage Pods

- ✓ ReplicaSets maintain desired replicas
- ✓ Scaling changes desired state
- ✓ Kubernetes constantly reconciles desired vs actual state